4.1.2 Using server actions for AI-powered RSCs

Server actions are a Next.js feature that allows you to run server-side code directly from your components. Think of them as server-side functions that you can call directly from your client components (like form elements), eliminating the traditional need for API routes. Let's explore how this magic works.

Here's what we're going to build:

- A server action that showcases how to process user input, interact with the OpenAI API, and return AI-generated responses without the need for explicit API routes
- An updated client code that renders the streamed updates from the server into a UI component that displays the current chat history

NOTE The source code for the examples in this chapter is available at https://github.com/Generative-AI-Web-Apps/Code/tree/main/ch04/chat-rsc.

Project code: chat-rsc

The code for this example project is in the ch04/chat-rsc folder. You can use that example by running this command in the root folder of the project repository:

```
$ npm run dev -w ch04/chat-rsc
```

You'll also need a Google API key to access their REST API. The appendix at the end of this book offers instructions on how to set up and create a secret key for Google AI and other providers:

```
// .env
GEMINI_API_KEY=<SECRET_KEY>
```

For details on setting up and creating API keys, refer to the appendix.

We'll examine the basic implementation of AI-powered RSCs using the Vercel AI SDK. We'll use two main files: a server component file for AI processing and a client component for user interaction.

In this example, we're creating a system that maintains a conversation thread with an AI model. The AI will respond to user inputs based on the conversation history using streaming responses.

Before we dive into the code, let's visualize this communication with our AI model using RSCs. Figure 4.4 shows the flow of data from user input to the final display of the AI's response.

As you can see, user input travels from the client component to the server action, which then interacts with the AI provider. The AI's response is streamed back in chunks, updating the client in real time while maintaining the conversation state in memory. Listing 4.1 shows the code that implements this process.